ARK-487 — Authority Engine · Cold Start Latency

Status: PREREGISTRATION (locked before execution)
Experiment Date: 2026-07-18
Series: ExecutionProof P02 Latency/Throughput/Scale

Question

What is the cold-start latency — time from engine construction to first correct authority decision — of the reference **Authority Engine**?

The Authority Engine answers the AUTHORITY half of Verification-Before-Execution: “Does this principal CURRENTLY hold the authority they are claiming, at the moment of execution — not merely at approval time?” ARK-483 measured cold start of the Verification Decision guard (the binding-match half). ARK-487 measures the analogous cold start of the current-state authority check.

Scope & Honesty Boundary

The component under test is a **deliberately minimal, in-process reference implementation** (`engine/authority_engine.py`) built to MEASURE component performance. It is **NOT** the production Authority Engine. A production engine (persistent store, replication, external IdP integration) is out of scope for these performance testbeds and belongs to a governed customer integration. All claims are bounded to this in-memory reference under the stated load.

Hypothesis

Cold start (construct reference engine of 1,000 principals × 10 grants, then first decision) completes with **p95 ≤ 50,000 µs (50 ms)**.

Component Under Test

Reference AuthorityEngine — exact, fail-closed authority check:
1. principal lookup (dict)
2. grant-tuple membership test (set)
3. current-state revocation test (set) — the VBE “now” check

ALLOW only if the grant exists AND is not currently revoked. No normalization, no case folding, no subset reasoning.

- **Implementation:** Python (`engine/authority_engine.py`)
- **Harness:** `engine/perf_harness.py --dimension cold_start`

Methodology

Test Design

- **Runs:** 200 independent cold-start cycles

- Each run: construct a fresh reference engine → perform one authority check → record elapsed time (μs)
- **Correctness gate:** before measurement, the engine must ALLOW a valid grant, DENY a mutated grant, and DENY a revoked grant

Metrics

Primary: `cold_start_us` — mean, median, p95 (μs)
Secondary: min, max

Thresholds (Gate Conditions)

C1 (Latency): p95 cold start ≤ 50,000 μs
C2 (Correctness): correctness gate must PASS (valid→ALLOW, mutated→DENY, revoked→DENY)

Verdict: PASS if C1 ∧ C2; otherwise FAIL

Kill-Gate / Effectiveness Conditions

K1: If the correctness gate fails (any unjustified ALLOW or wrong DENY) → GATE-STOP; performance numbers are void.

Honest Findings Commitment

Any deviation from hypothesis (higher/lower than predicted, correctness failures) will be disclosed in RESULTS.md. If the correctness gate fails, the experiment is reported as GATE-STOP regardless of latency.

Execution Protocol

1. **Lock:** Commit this preregistration + compute MANIFEST.txt hashes
2. **Execute:** Run `engine/perf_harness.py --dimension cold_start`
3. **Record:** Save results to `results/coldstart_results.json`
4. **Evaluate:** Compare against thresholds C1, C2
5. **Report:** Document findings in RESULTS.md with honesty

Preregistered: 2026-07-18
Investigator: Remnant Fieldworks Inc. / ExecutionProof Research Program
Covenant: RF Standing Covenant (preserve outcomes, bound claims, honest disclosure)